For my independent study project, I created StepOut, a route-based exercise app designed to help people fit movement into their busy schedules. The whole idea was to remove the friction from getting outside and moving around. I've noticed that a lot of people (myself included) have these small windows of free time like a lunch break or a gap between classes, but we don't want to spend half of it just planning where to go. StepOut solves that by letting you put in a time or distance, pick what kind of route you want, and boom you have now got a route. The random generation feature is honestly my favorite part because it completely removes the decision-making process making getting my steps in just that little bit eaiser.

The biggest thing I learned from this project is how different it is to build an actual full app versus doing smaller class assignments. In most of my classes, you're working on one isolated concept at a time. But with StepOut, I had the map system, route generation, the UI, saved routes, pace settings, widgets, a tutorial system, and animations, and they all had to work together. Even when each piece worked fine on its own, putting them together created a whole new set of problems. I realized that software development isn't just about making code that runs it's about making all these systems talk to each other in a way that makes sense.

Throughout the project, I got hands-on experience with a bunch of Apple frameworks I'd never really used before. I worked with MapKit to place pins, generate routes, track location, and draw overlays on the map. I used Core Data to save and reload route information. I even got into Live Activities and widgets so users could see their progress without opening the app. I'm still learning all of this, but now I understand how these pieces fit into a real application instead of just reading about them in documentation.

Some parts were honestly easier than I expected. Setting up Core Data for storing routes was intimidating at first, but once I wrapped my head around how it's structured, it wasn't that bad. The basic stuff like drawing routes and placing pins was also straightforward. But then things got complicated fast. Making route generation reliable and flexible enough to handle different use cases turned out to be way harder than I thought.

The most challenging part of the whole project, by far, was the user interface. I went into this thinking routing logic would be the hardest thing to figure out, but UI development ended up taking the most time and the most iterations. Things like slide-out panels, route settings, pacing controls, color themes, and the tutorial overlay required constant tweaking. And it wasn't just about making features exist they had to make sense to someone who'd never used the app before.

When I started testing with other people, the biggest issue wasn't that features were broken it was that people didn't know they existed. They'd ask, "can it do X?" and I'd be like "yeah, it's right there," but they just hadn't noticed the button or didn't understand what it did. That's when I realized I needed to add the tutorial system.

Another huge challenge was the logic behind route generation itself. The app supports one-way routes, out-and-back routes, and loops, plus you can input either distance or time. Then I added pace-based customization on top of that, which made everything even more complex. For example, if you generate a 30-minute route for walking versus running, those should be totally different routes even though the time is the same. This meant I had to keep calculations consistent across route generation, progress tracking, pacing logic, and the display and I learned really fast how one small change in the logic could break things in three other places.

One of the most important lessons I took away is that usability is just as important as functionality. You can build the most powerful feature in the world, but if users can't figure out how to use it, it's basically worthless. That shifted my focus from just adding more features to making the existing one’s clearer layout, better labels, better feedback, and better onboarding. I also learned how to interpret user feedback better. Sometimes people tell you exactly what's wrong, but other times their feedback is pointing at a deeper design issue you didn't even notice.

I also gained a much better understanding of how much time polish takes. Animations, color themes, button feedback, little visual details they might seem minor, but they make a huge difference in how the app feels. That's what took it from "this works" to "this actually feels like a real app." It taught me that polish isn't just decoration it's part of making something usable.

Another big part of this project was learning how to use AI as a development tool. I used it to help with debugging, implementing UI stuff, and exploring parts of the frameworks I wasn't familiar with. But I still had to define what I wanted, test everything, evaluate whether the solutions worked, and make all the design decisions myself. AI sped things up a lot, but it didn't replace needing to understand what was going on. This is especially true for “flare” implementations like animations. You still have to know enough to tell when something's wrong or when a solution doesn't make sense.

Working on this independently also made me realize how valuable team-based development is. Doing design, logic, testing, and debugging all by myself gave me a lot of

freedom, but it also made it way harder to keep quality high across everything. It helped me understand why real software projects teams have people who specialize in different areas.

The most rewarding part of the whole thing was watching it all come together. Routing, data storage, pacing, and the interface started as completely separate pieces, and seeing them work together as one cohesive app was honestly really satisfying. It stopped feeling like a class project and started feeling like an actual product.

Overall, StepOut gave me a much clearer picture of what it actually takes to build a full application. I developed technical skills in iOS development, MapKit, Core Data, and UI design, but I also learned broader lessons about iteration, usability, and how to make systems work together. More importantly, I learned how to take an idea and turn it into something functional that people can actually use.